

Project Report: SNV Functional Impact Classification Using Deep Learning

Introduction

Single Nucleotide Variants (SNVs) are the most common type of genetic variation, where a single nucleotide base is replaced by another in the DNA sequence. Classifying SNVs as **deleterious** (likely to cause disease) or **neutral** (harmless) is vital for understanding genetic disorders and improving clinical diagnostics.

Objective

This project aimed to develop an end-to-end deep learning pipeline to classify SNVs based on their functional impact using public variant data and sequence-based features.

Methodology

Development Approach:

The pipeline was built iteratively, identifying and resolving model weaknesses step-by-step.

1. Data Acquisition:

- a. Downloaded ClinVar VCF file (`clinvar.vcf.gz`) from NCBI.
- b. Focused on SNVs with clear clinical significance annotations.

2. Feature Extraction:

- a. Parsed VCF lines for chromosome, position, REF, ALT, allele frequency, and clinical significance.
- b. One-hot encoded nucleotide bases.
- c. Assigned binary labels: `deleterious` → 1, `neutral` → 0.

3. Model Training:

- a. Built an Artificial Neural Network (ANN) using Keras.
- b. Applied **class weights** to handle data imbalance.
- c. Added training callbacks: `EarlyStopping` and `ReduceLROnPlateau`.

4. Evaluation:

- a. Used metrics: Accuracy, Precision, Recall, F1-score.
- b. Visualized training curves and confusion matrix.

Model Development Journey

Attempt 1: The "Lazy" Model

- Used only one-hot encoded REF and ALT bases.
- Issue: Model always predicted "neutral" due to class imbalance.
- Accuracy was ~87% but Recall for deleterious = **0%**.

Attempt 2: Class Weighting

- Introduced `class_weight` in model training.
- Model began identifying harmful variants.
- Result: Accuracy = 68%, Recall = 45.2%.

Attempt 3: Final "Smart" Model

- **Feature Engineering:** Added allele frequency.
- **Deeper Architecture:** 3 hidden layers with dropout.
- **Smart Training:** Used callbacks to prevent overfitting.
- Final Accuracy: 69.8%, Recall: 45.2%

Extracted Features

- **chrom, pos:** Chromosome number and position of the variant
- **ref_A, ref_C, ref_G, ref_T:** One-hot encoding of the reference (REF) nucleotide base.
- **alt_A, alt_C, alt_G, alt_T:** One-hot encoding of the alternate (ALT) nucleotide base.
- **allele_freq:** Population frequency of the alternate allele; rare variants (low frequency) are more likely to be deleterious.
- **label:** Target variable indicating the variant's impact: 1 for *deleterious*, 0 for *neutral*.

These features allow the model to learn from both the type of nucleotide change and how common that change is in the population.

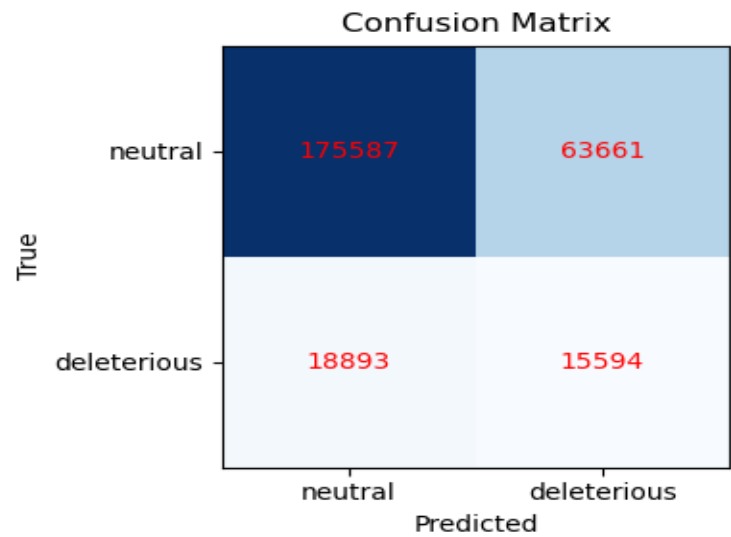
Results & Analysis

Evaluation Metrics:

Metric	Score
--------	-------

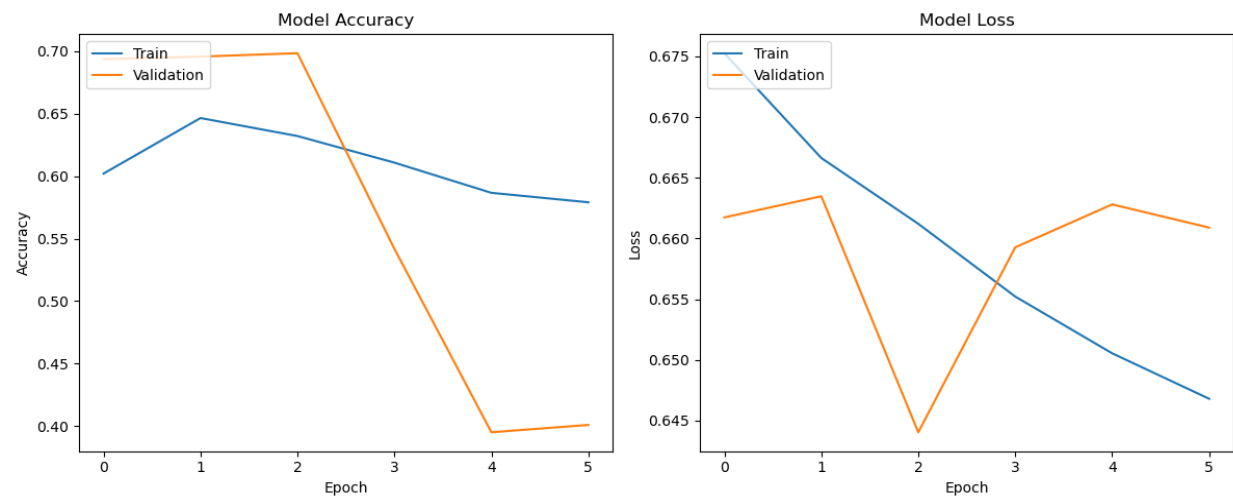
Accuracy	69.84%
Precision	19.68%
Recall	45.22%
F1-Score	27.42%

Confusion Matrix:



Model successfully identified **15,594 true positive** deleterious variants — a huge improvement over the initial model.

Training History:



Training stopped automatically at epoch 6, thanks to `EarlyStopping`, just when validation accuracy stopped improving — saving time and preventing overfitting.

Tools & Libraries Used

- **Python**
- **pandas, numpy** (Data handling)
- **scikit-learn** (Train-test split, metrics, class weights)
- **Keras/TensorFlow** (ANN model)
- **matplotlib** (Visualization)
- **requests, gzip** (Data download and parsing)

Limitations

The model relies only on sequence and allele frequency data. Incorporating more **biological context** such as protein structure, conservation scores, or gene expression data could further improve classification accuracy.

Conclusion

This project successfully delivered a full deep learning pipeline that classifies SNVs based on their functional impact. Through iterative improvements, class balancing, and thoughtful feature engineering, I built a model capable of identifying truly harmful variants, which is valuable for medical diagnostics.

To further improve performance:

- Add biological features (e.g., SIFT, PolyPhen, conservation scores).
- Try advanced architectures (e.g., 1D-CNNs or Transformers).

Or try some more improvements...

